Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

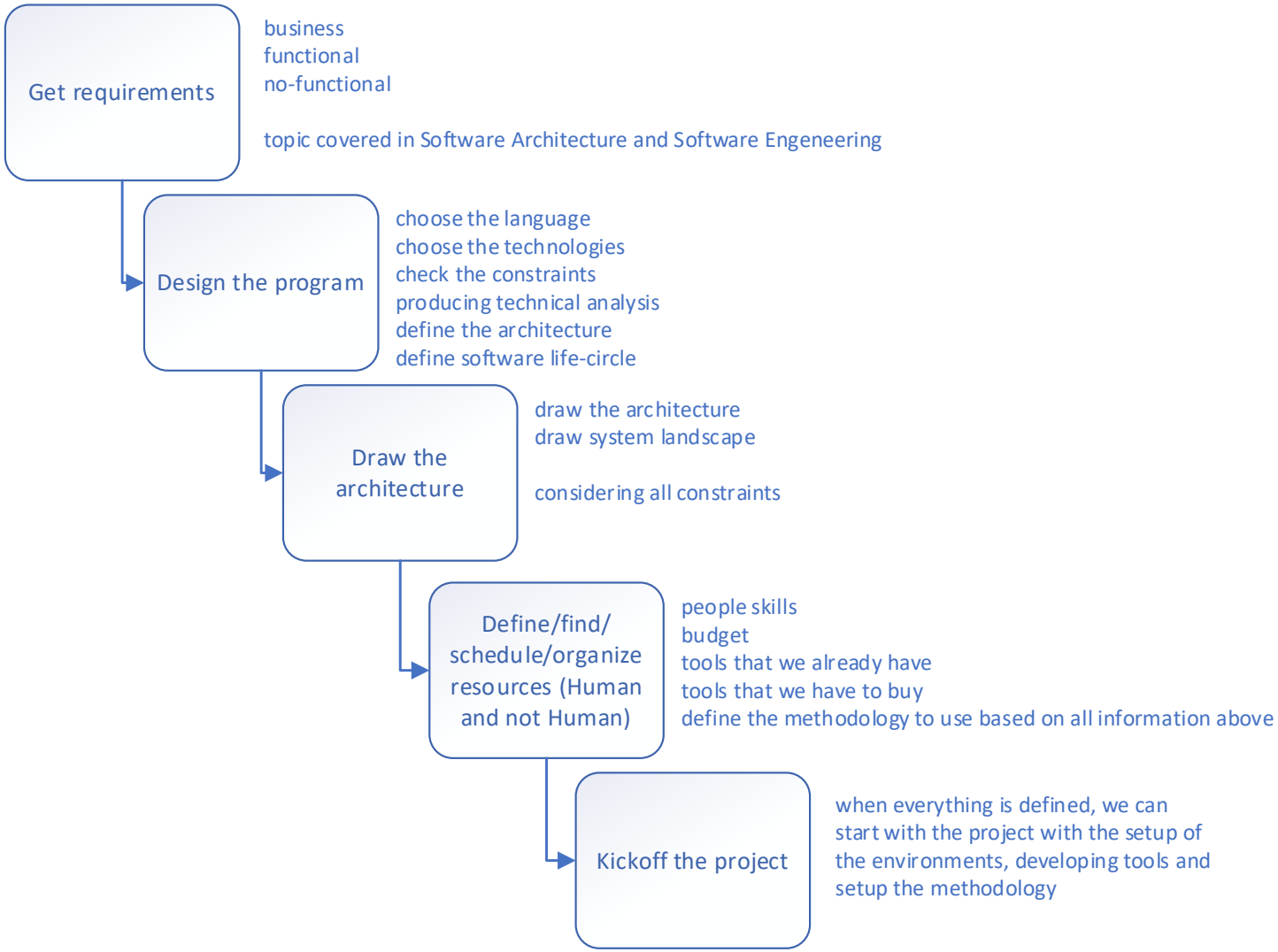
Starting with the basis

- Managing software projects has nothing at all to do with programming
- Nobody has a degree in Managing Software Projects and, as already said, there are very few books on the subject
- Making software is not a manufacturing process
- Writing code is not *production*. It's not always craftmanship. It's *design*. Design is that nebulous area where you can add value faster than you add cost.

source: Joel on software book

Ca' Foscari University of Venice

Overview the process



Ca' Foscari University of Venice

How take the right choice?

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
MGPNNL

Copy participation link

Key points to consider

- Remember: this choice will follow you basically forever
  - you do not believe me?
  - think about Cobol and bank application or legacy ERP
- Look the requirements
- Look for the support and maturity of the technology
- Look for the characteristics of the technology

Possible constraints

- Security aspects
  - OWASP - https://owasp.org/
  - Hardening - enhancing security by reducing vulnerabilities through configurations, updates, and removing unnecessary features
  - Privacy and security by design
- Performance in run time (python vs java)
- Performance in deployment (java vs php)
- ...to be continued

Possible constraints

- Business constraints
  - agile methodology applied to the project
  - low availability of the contractor
  - internal policies (e.g., some systems cannot call other systems, they can only receive call)
  - starting from a MVP and fast delivery of new features
  - a "big-bang" deployment for the project

Ca' Foscari University of Venice

A real case: an E-commerce project

- ICT team
  - E-commerce
    - Backend
    - Frontend
  - CRM
  - ERP
  - OMS
  - Middleware
  - Infrastructure
  - Project vs maintenance
- Security
- Business
  - HQ vs local
  - Content
  - GA/SEO
  - Designer
- Logistics
- Legal

Business constraints: Agile scrum methodology

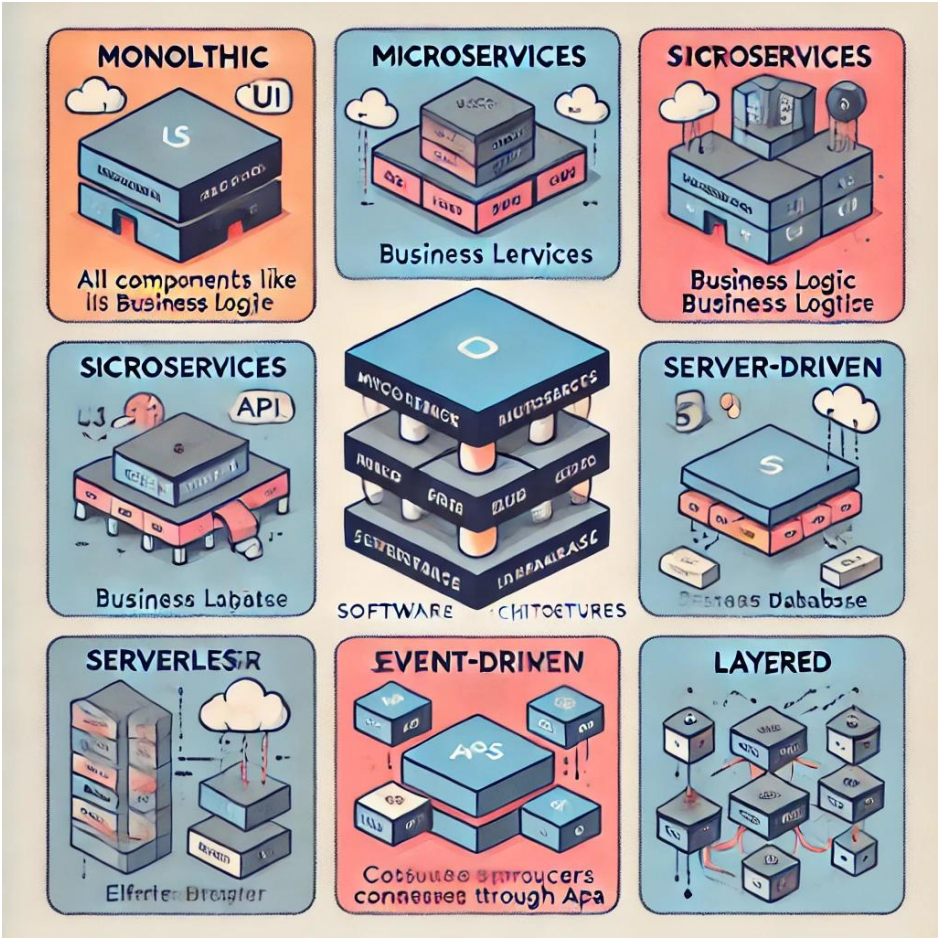
it's a high arity function

- we need to put on the table all the variables
- keep it simple - Occam's razor
- look the market
  - e.g., Java and .Net are the two program language for enterprise application: maybe you can do the same thing on Go-lang, but why do you want introduce any risks?

Ca' Foscari University of Venice

Choosing architecture

- think about your need
- do not force architecture (e.g., a single page web site hosted on a microservice architecture)
- think to the future, not only look for today issue



a real case: a DAM

- DAM: Digital Assets Manager
- Needs: scalability and possibility to extend features
- PIMCore - https://pimcore.com
- Chosen architecture: container
- Then we can use Kubernetes - https://kubernetes.io/

- ...very cool! but...

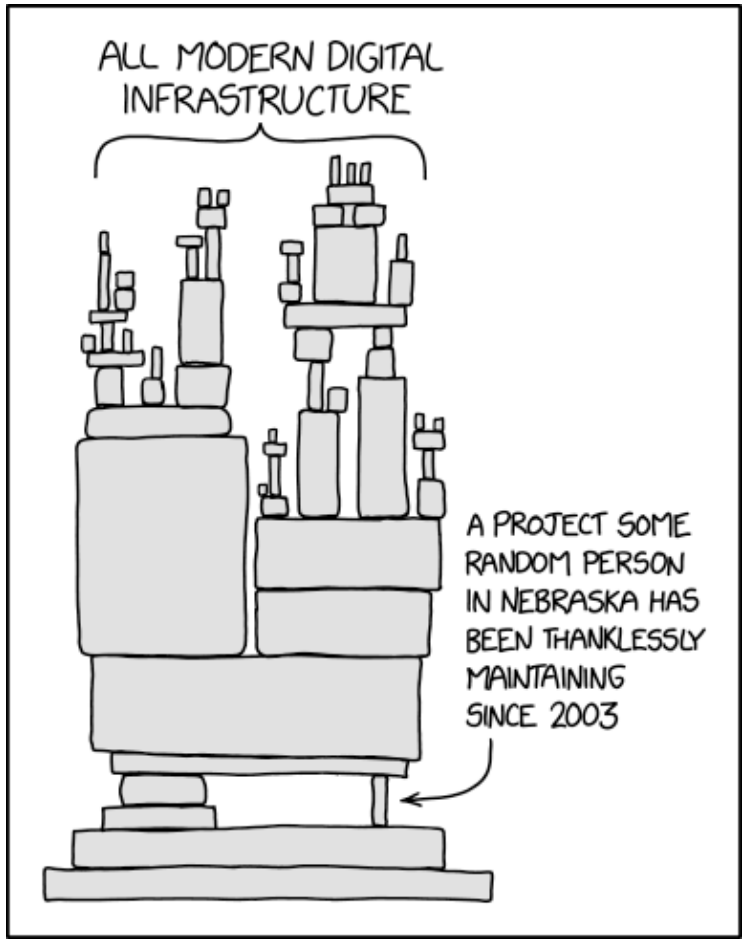
a real case: a DAM

- PIMCore is thought for a monolithic architecture
- More than 50 days spent (wasted?)

- Nobody make a previous check on the product
- Nobody rise the hand and said: it does not work

- Take away: rise your hand if you find something that does not work properly (better to be safe than sorry)!
- In Italian: "Meglio arrossire prima che piangere poi"

and choose carefully external components



source: https://xkcd.com/2347/

Ca' Foscari University of Venice

a real case: polyfill.io

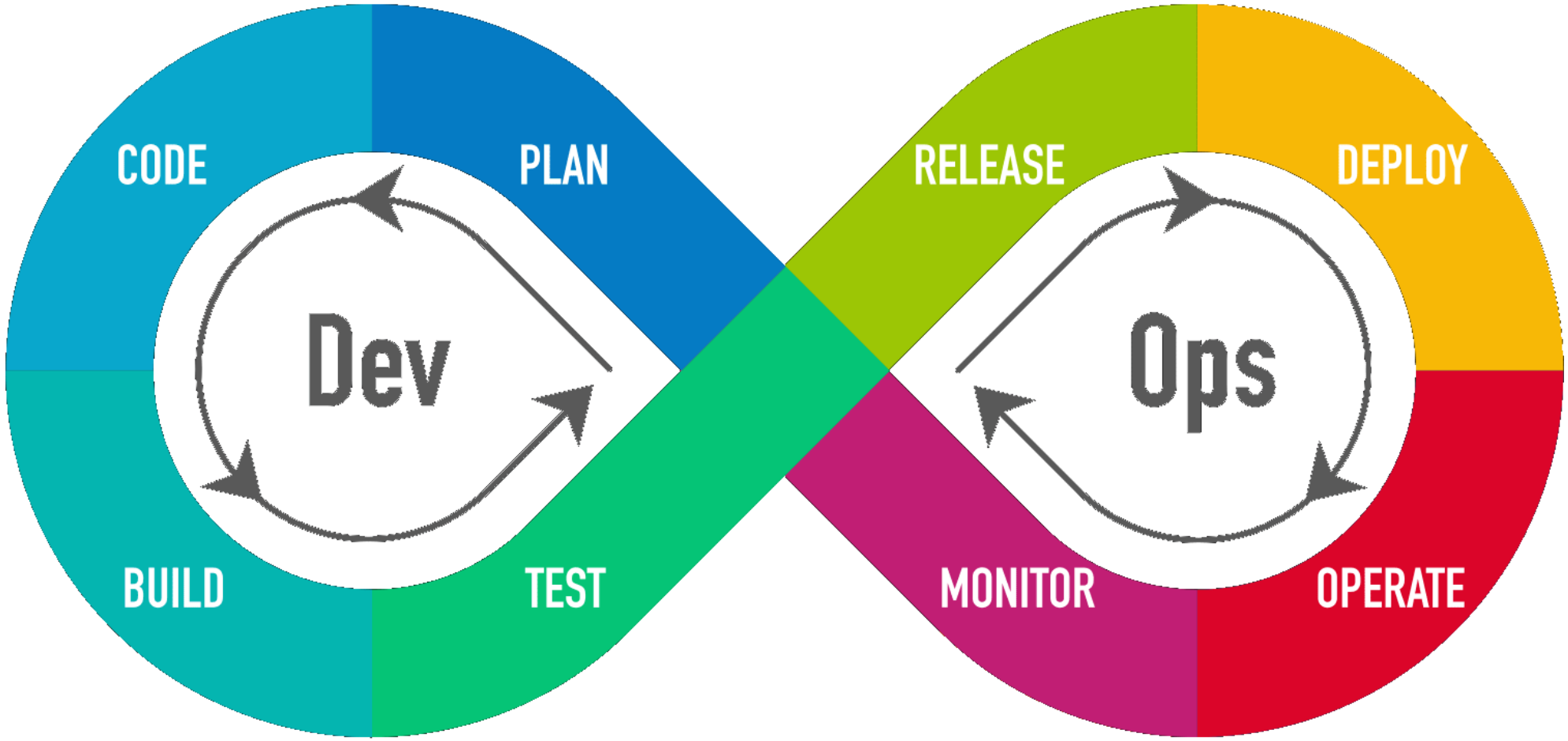
- Frontend library for IE11 compatibility (written in JS)
- Used by some components
- On January 2024 "Funnull" (Chinese CDN) bought the property of this library
  - Funnull, usually, hosts porn sites and gaming sites
- In June 2024 some issues rise
  - backdoor in library was found
  - third-party domains blocks rose
  - google blocked adv for website that use this library

some links:

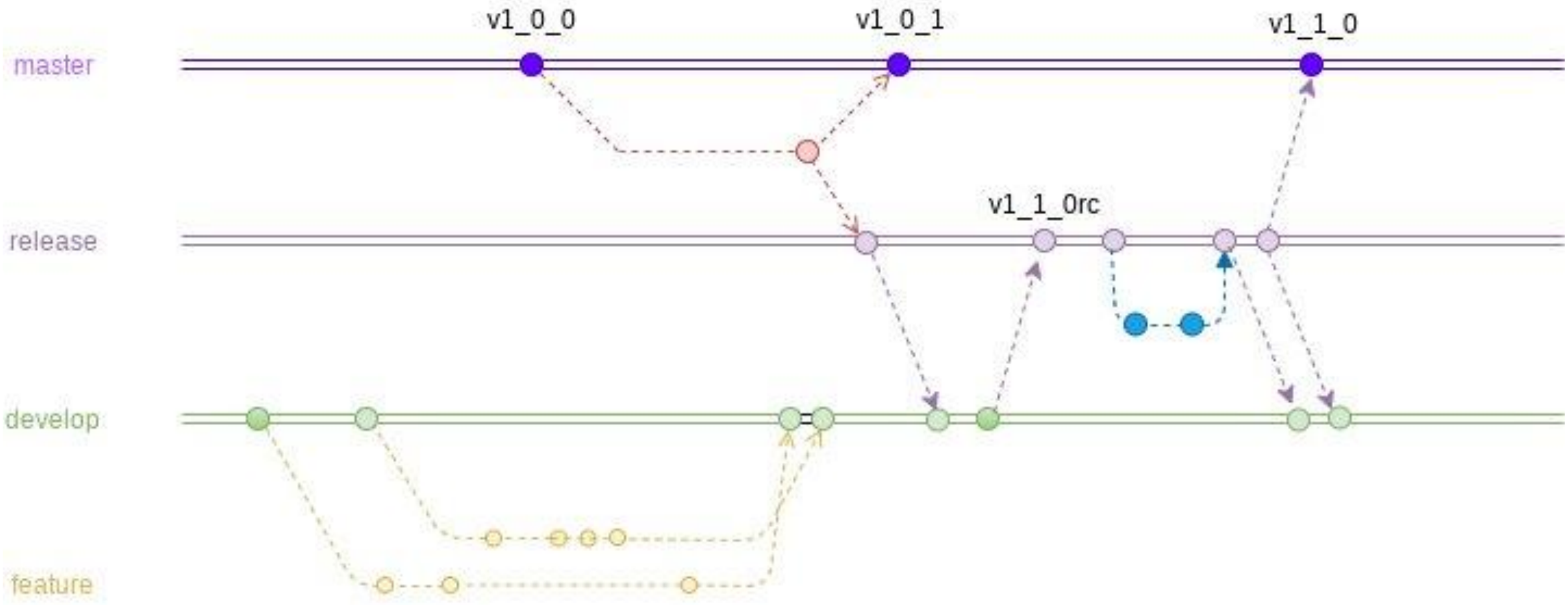
- https://tinyurl.com/29neqzo2
- https://tinyurl.com/28gq6sp7

Ca' Foscari University of Venice

Software lifecycle

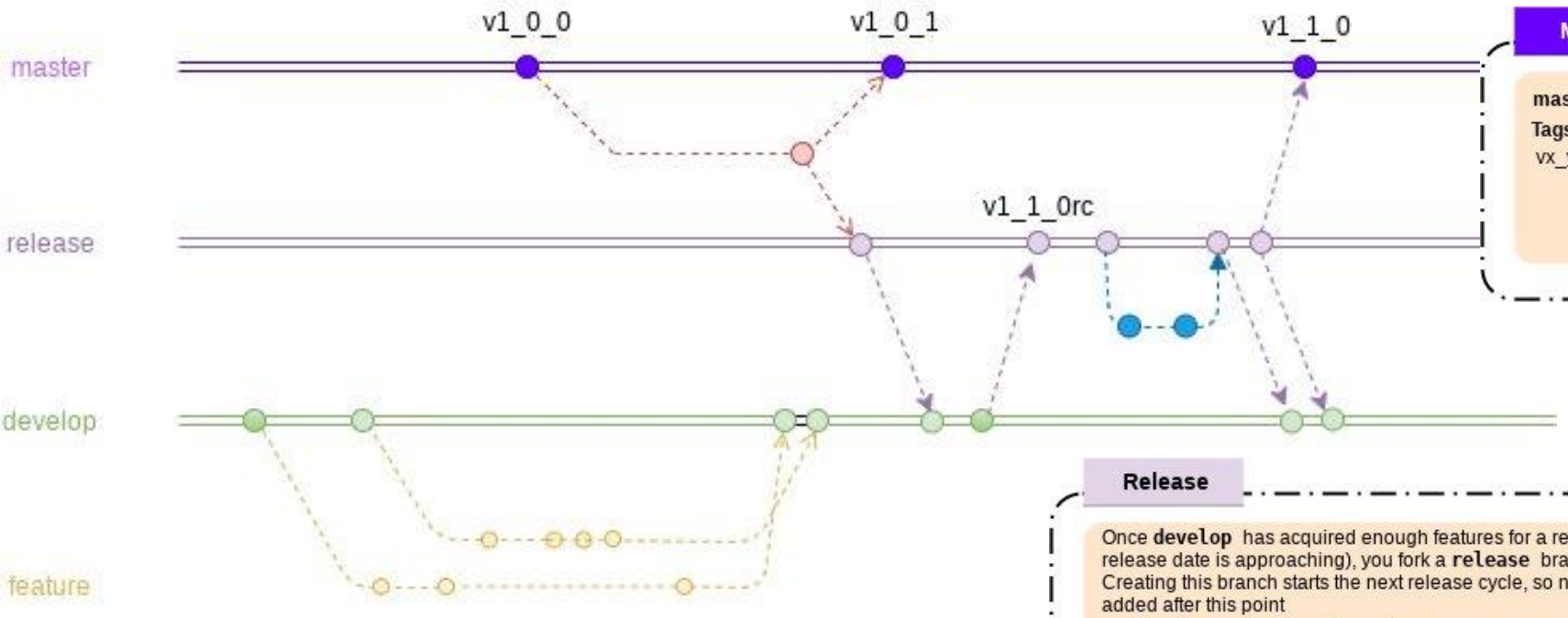


Gitflow



Ca' Foscari University of Venice

Gitflow



Master

master branch always stays intact (except for hotfix).

Tags naming convention:

vx_y_z I.E: v1_2_0

v: short form of **version**

y: minor version (sprint)
z: hotfix incremental

Release

Once **develop** has acquired enough features for a release (or a predetermined release date is approaching), you fork a **release** branch off of **develop**. Creating this branch starts the next release cycle, so no new features can be added after this point

Naming convention: release/vx_y_zrc
Example: release/v1_1_0rc

Once the **release** is ready it is merged to both **master** (and tagged with a **version number**) and **develop**.

Feature

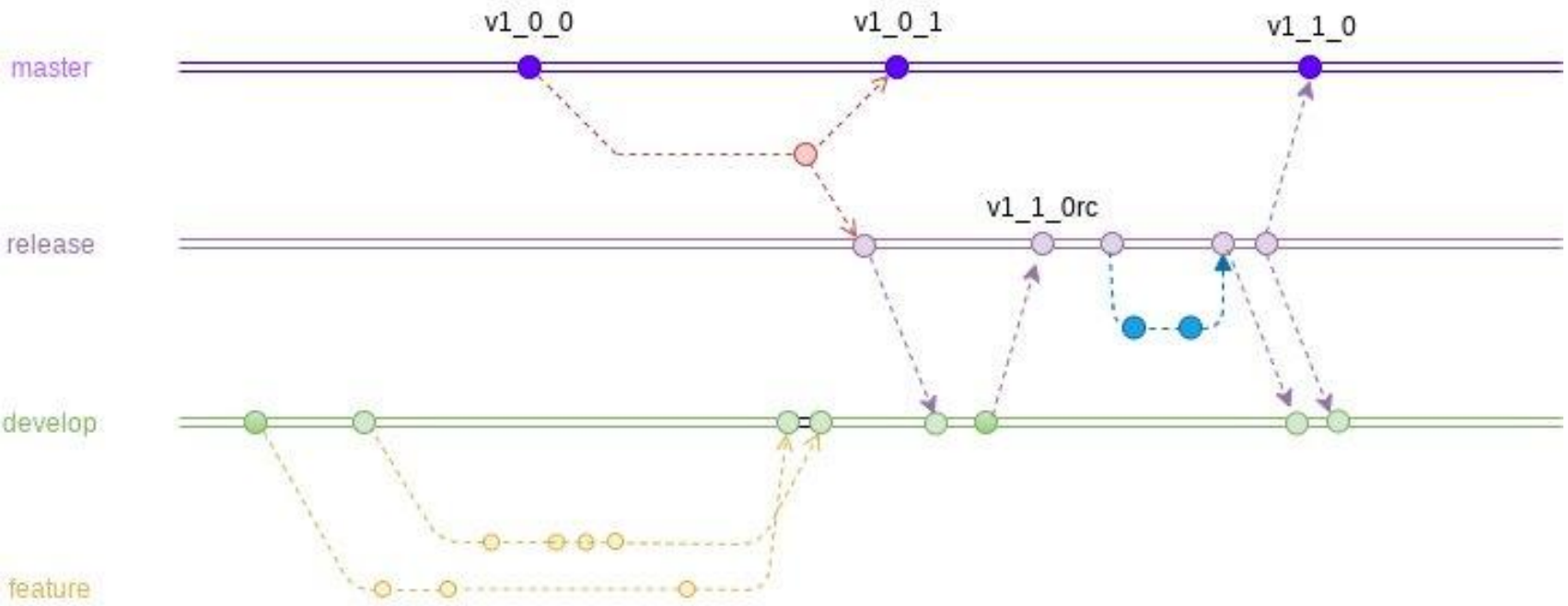
Each new feature should reside in its own branch, **feature** branches use **develop** as their parent branch

Naming convention: feature/nsurname/issueid-brief-and-usefull-name
Example: feature/pbordin/0022-add-guest-checkout

Once the **feature** is complete, it is merged back into **develop.**
Remember to merge frequently develop into feature branch to avoid/resolve conflicts.
Features should never interact directly with **master**.

Ca' Foscari University of Venice

Gitflow



Bugfix

No new features —only bug fixes, documentation generation, and other release-oriented tasks.
(Non-urgent bugfixes can be created as features, merged to the develop branch and released later)

Naming convention: bugfix/nsurname/issueid-biref-and-usefull-name

Example: bugfix/pbordin/0024-fix-alignment-on-pdp-images

Once the **`bugfix`** is complete it is merged back into the release branch and develop.

Hotfix

If an issue in **`master`** is detected a **`hotfix`** branch is created from **`master`**

Naming convention: hotfix/nsurname/issueid-brief-and-usefull-name

Example: hotfix/pbordin/0023-change-production-endpoint

Once the **`hotfix`** is complete it is merged to **`release`** and **`master`**.
After that, release branch will be merged into develop

Code review

Code reviews look, to me, like an imperfect solution to two *supremely important* problems:

1. ⏳ **Keeping code quality high** — which leads to ease of change, fewer defects, less maintenance, you name it.
2. 📚 **Sharing knowledge across the team** — which creates alignment, growth, resilience, flexibility, and more.

These are also *self-reinforcing* goals: shared knowledge helps keep quality high, and, in turn, high quality code is generally crisper and easier to understand for others.

Today, the most common workflow for code reviews is async + blocking + mandatory, that is:

- 🔀 **Async** — a PR is opened, one reviewer (or more) is assigned, and they will perform their review asap.
- ⛔ **Blocking** — code isn't merged and deployed until it passes the review.
- 🔒 **Mandatory** — you do the same workflow for all changes.

- Code review is your main (and possible only) feedback loop on how your team writes code
- Intercept defect and alignment on practice and culture

source: https://refactoring.fm/p/thoughts-on-code-reviews

Code review

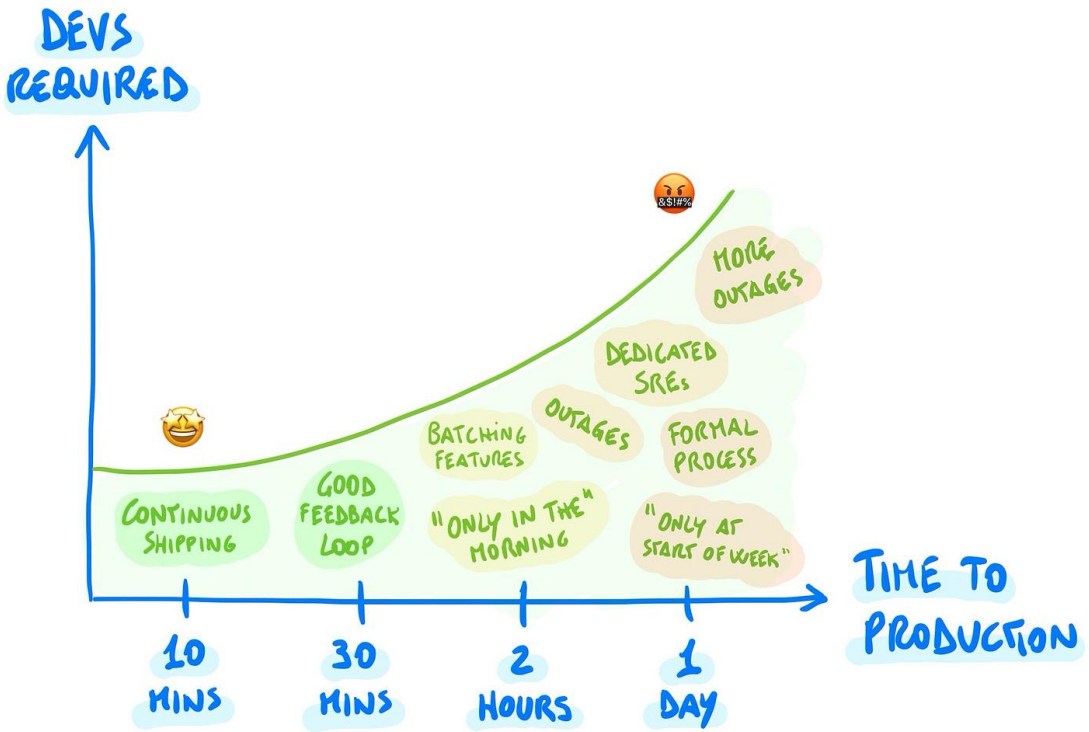
- What happen if you do not plan the review (maybe for a team of senior devs)?
- 3 main issues
  - Gatekeeping (over time, each engineer had developed their personal areas of ownership, which had become impenetrable to everyone else)
  - Inconsistency (different parts of the code had completely inconsistent choices about naming, libraries used, folder structure, and more)
  - No docs (since the general expectation was that everyone worked on their own, there was little incentive to write good docs and keep them updated)
- 4 secondary issues
  - Collaboration (any conversation about design, tradeoffs, or estimates, was extremely hard, because everyone's ownership was completely siloed)
  - Hiring (onboarding new engineers, either junior or senior, was a nightmare)
  - Resource allocation (if we wanted to invest more in a specific product area, it was hard because we couldn't put more people on it at will)
  - Key man risk (individual engineers got an outsized importance for the business and, even if—to their credit—no one ever tried to take advantage of it, it was concerning to me as a manager)

source: https://refactoring.fm/p/thoughts-on-code-reviews

Ca' Foscari University of Venice

Code review

- What about slow or superficial reviews?
- Superficial -> same scenario as before
- Once a developer fires up a PR, their work is far from over: they may need to make improvements (based on the review), and, when the feature is released, check that everything works fine in prod through logs and instrumentation.
- But including several hours of delay at some point in this process (which you should also multiply by the number of review iterations) makes engineers switch to other tasks, which increases WIP, increases cognitive load, reduces productivity, leads to batched releases, and a whole self-reinforcing cycle of badness.



source: https://refactoring.fm/p/thoughts-on-code-reviews

Code review

> *I used to have hard opinions about commits and PR, and over the years, I have softened my stances.*
>
> *I used to see PR as a bug catcher and code quality. Now, I tend to watch PRs ask for knowledge transfer and paper trails.*
>
> *For code quality, I rely on linters, team code standards and internal tools. For bugs, on tests, types, and video walkthroughs.*
>
> *I have settled into trunk-driven development in the last 4-5 years. Most bigger features are split into multiple pull requests and shipped with a feature flag. To merge, one review is required. Often, this is someone who works closely with you on this feature.*
>
> *— Radoslav Stankov, CTO at Angry Building*

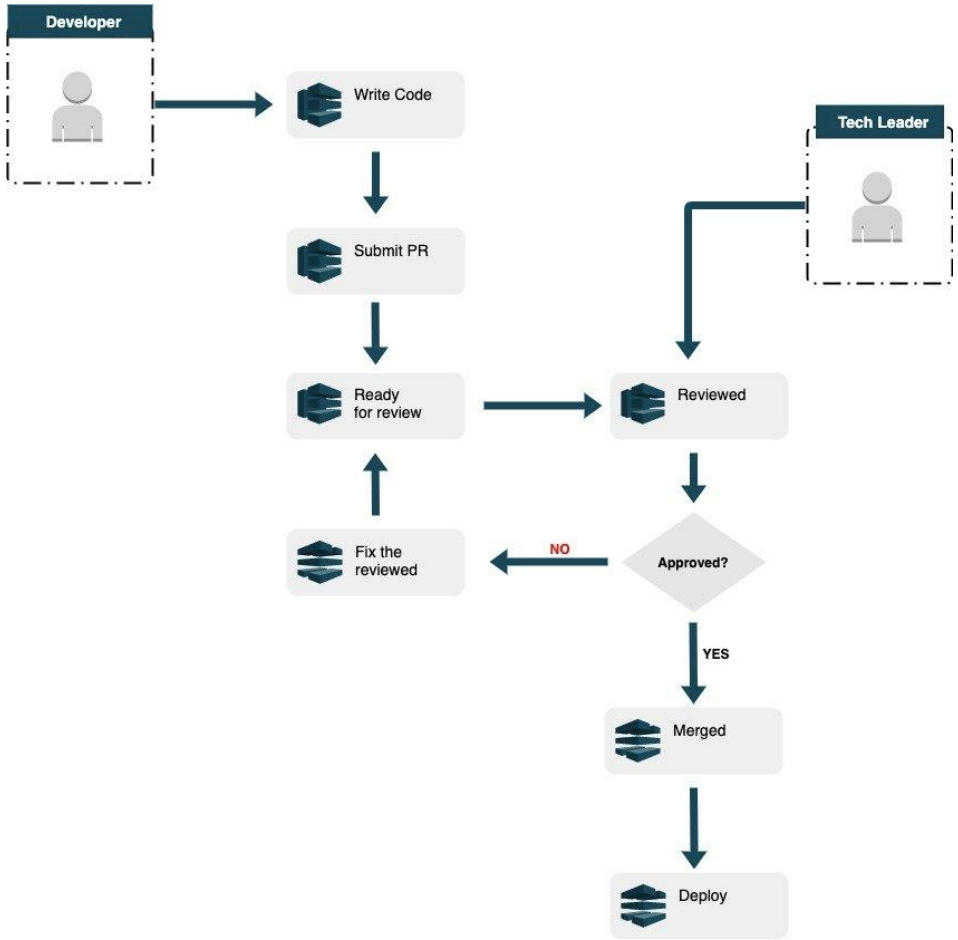
- 3 devices that impact how you write code:
  - Principles: coding practices & conventions that you have encoded over time
  - Design docs: where you discuss your design and implementation plan. You may follow a template that enforces good practices
  - Static analysis: where you catch coding standards violations, security vulnerabilities, missing tests, and various kinds of smells
- Review act as a "fallback"
- One of the goals of code reviews should be to continuously reduce their own scope, by allowing engineers to uncover items and rules that can be enforced by other parts of the dev process.

source: https://refactoring.fm/p/thoughts-on-code-reviews

Ca' Foscari University of Venice

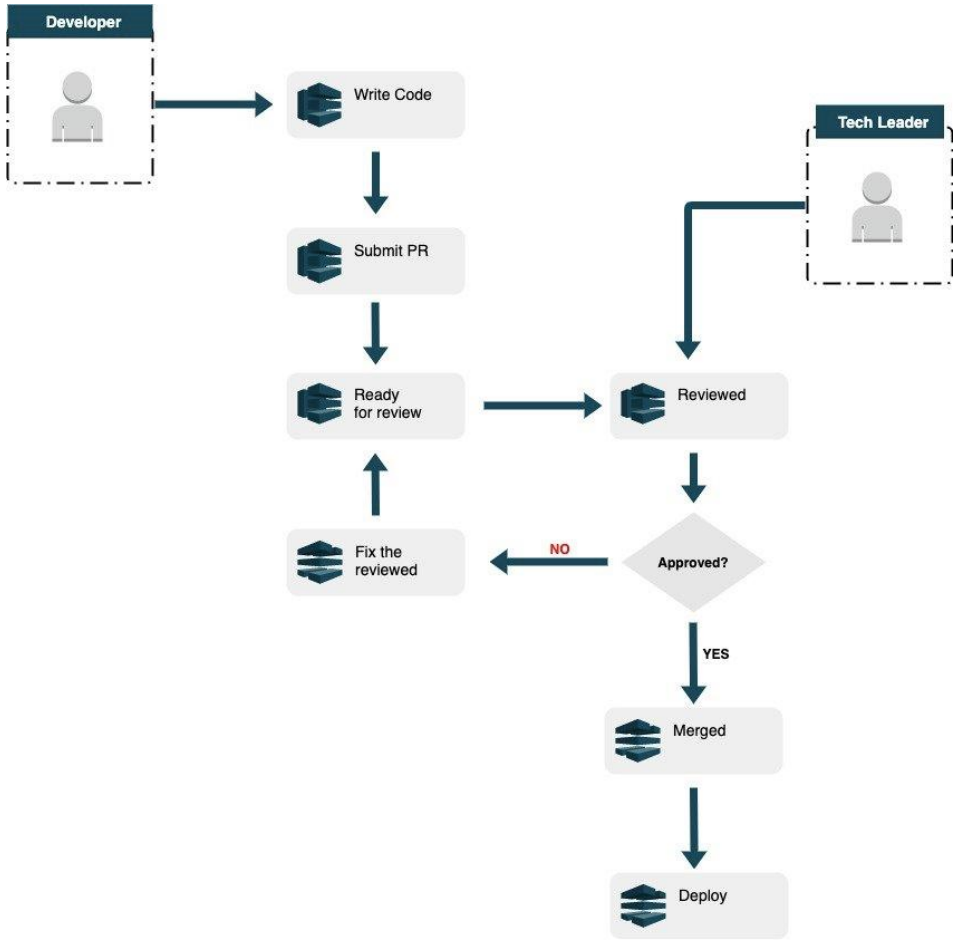
Code review

- Code quality
  - code passes style check
  - code is clear and concise
  - UI code is internationalized
  - Code has adeguate comments where appropriate
- Code is in an appropriate place
  - classes are in appropriate cartridegs
  - code fits with class's responsibilities (SRP)
  - configuration files are documented enough



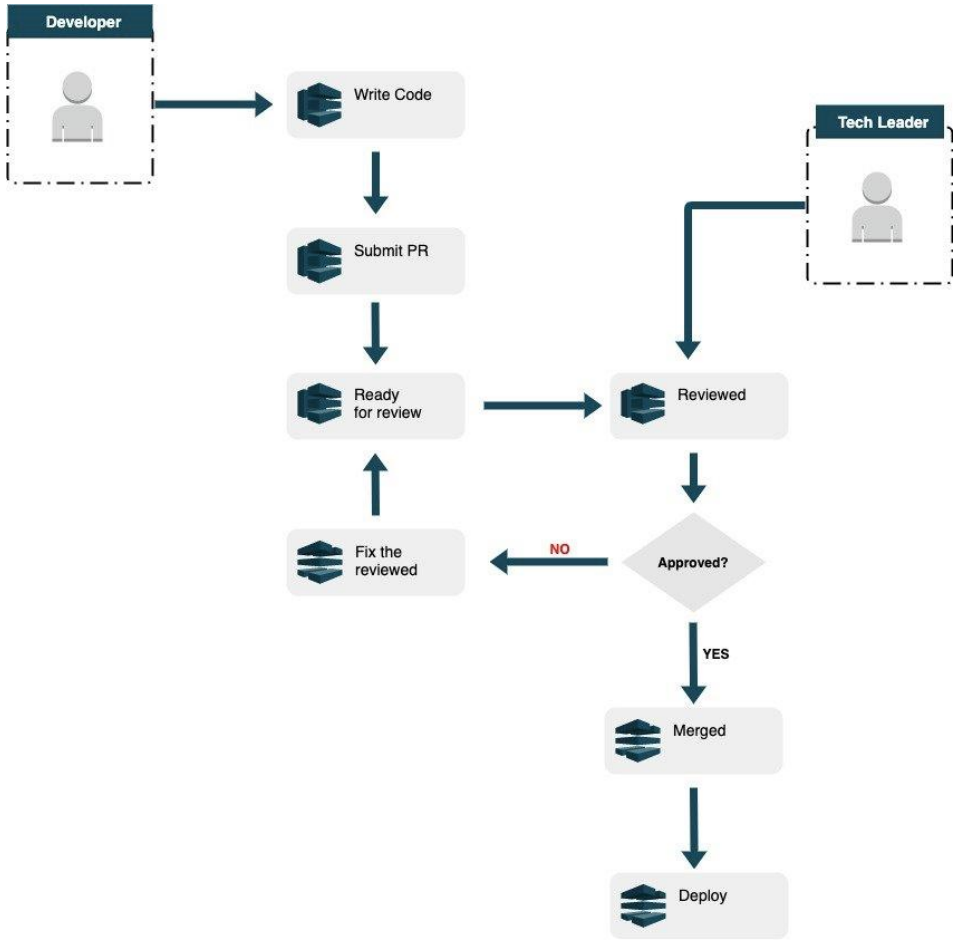
Code review

- If code is removed
  - no remaining code references the removed code
  - no orphaned config settings
  - no remaining comments refer to the removed code
  - unused imports and libraries are removed

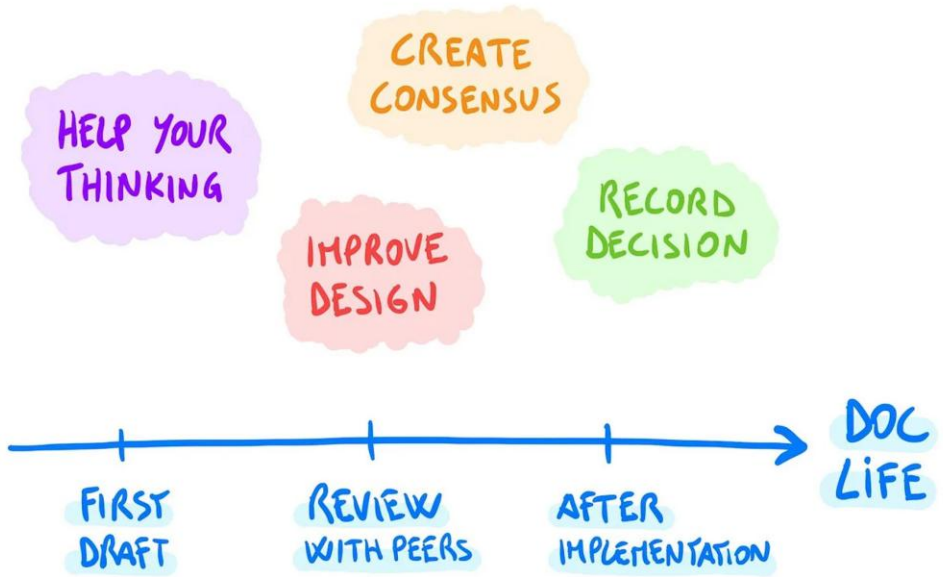


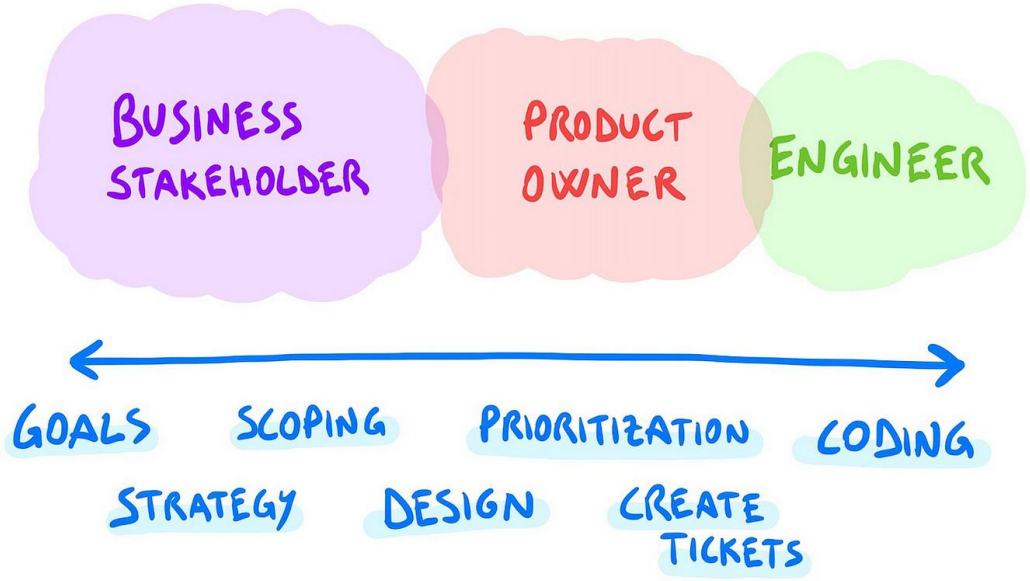
Code review

- Testing
  - new code has tests
  - changed code has tests
  - all tests are passed
  - test coverage stable or increased
- Documentation
  - new features are documented
  - docs removed for deleted features
  - docs updated for all changed features
  - devs/sysadmin doc updated if appropriate



Documentation lifecycle





source: https://refactoring.fm/p/continuous-docs-product-engineers

off-script slide

- Interview to DHH powered by Luca Rossi: let's watch it!
- David Heinemeier Hansson: creator of Rails (Ruby on Rails), co-founder & CTO of Basecamp and NYT bestselling author
- Topic: David's opinions on the current state of software development, including excessive complexity in software and infrastructure

off-script slide

- DHH blog post: https://world.hey.com/dhh/merchants-of-complexity-4851301b
- topic: companies that profit from complicated layering have an incentive to maintain it. A conflict between simplifying software and sustaining a business model
- Let's watch the first 40 seconds

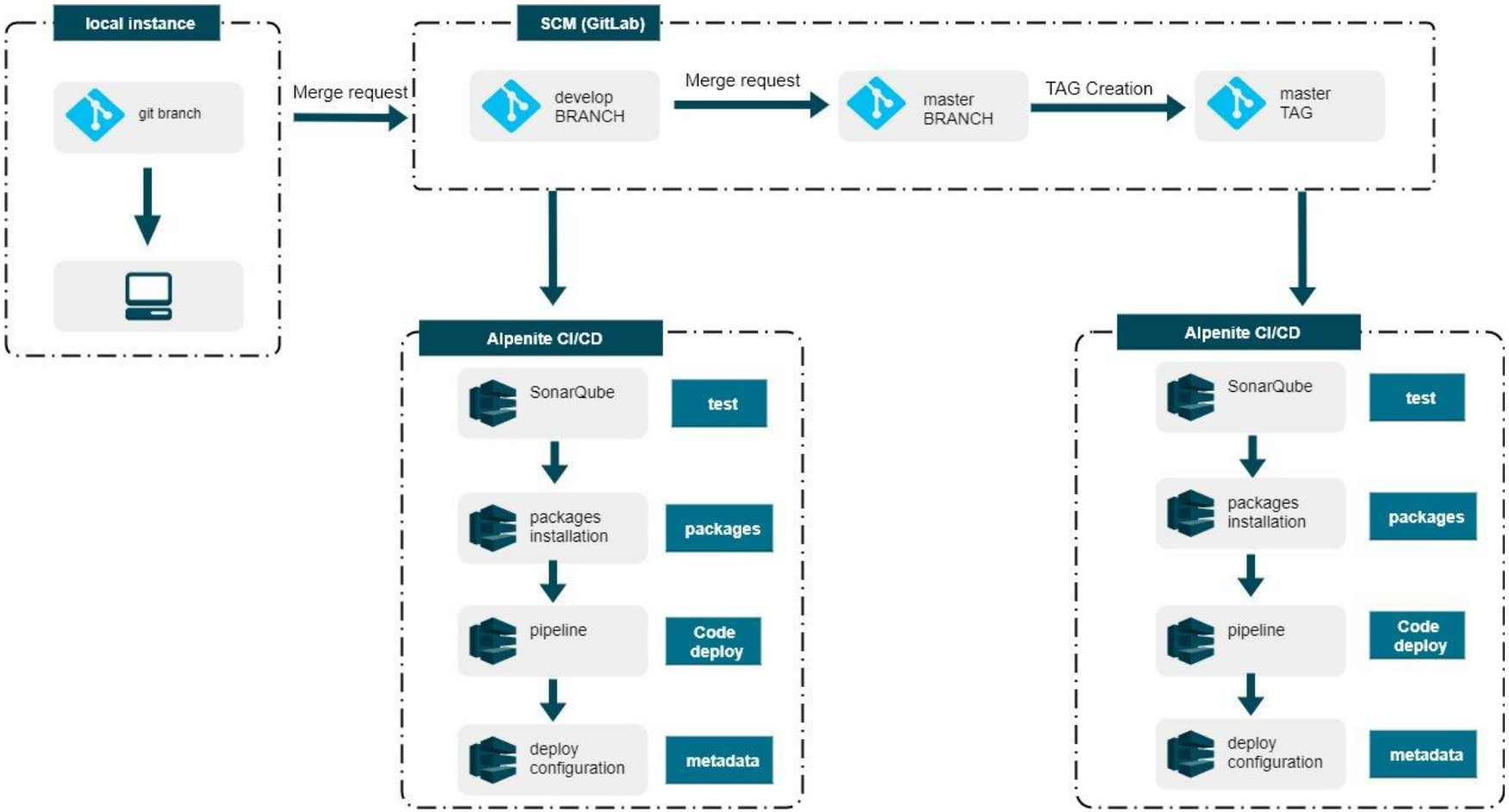
Ca' Foscari University of Venice

off-script slide – take away

- Reduce (or compress) the complexity always
- Investing in timeless knowledge
- Be aware on adding layer on layer ;-)

Ca' Foscari University of Venice

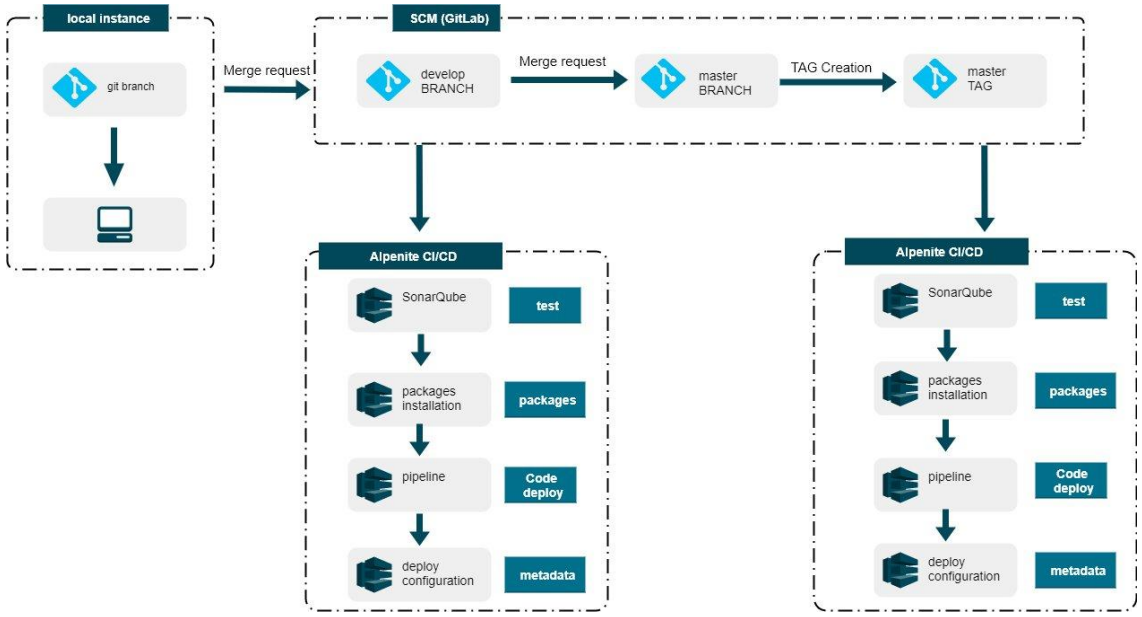
Development process – real case



Ca' Foscari University of Venice

Development process

- Environments: generally, we have 4 environments
  - Local instance/sandbox
  - Development
  - Staging/Test/Quality
  - Production



Same stack?

Ca' Foscari University of Venice

It works on my computer

- Consider versions of application stack (e.g., php6 works differently from php7)
- Consider versions of library
- Consider version of compiler/runner
- Consider to use same tool (Apache http server is different from ngnx, even if they do the same thing)

Development environment

- After you implemented and tested locally, you can open a PR in order to merge your work with the work of others
- Now you need an environment to perform the tests all integrated code (a.k.a. System Integration Test – SIT)
- Here you will know the regressions: when changing something, something else stops to work properly
- You have to re-test always everything when you deploy in the development environment
- Here you can still test with dummy content/data, here you can test all cases (even with all type of inputs)

Staging environment

- Ok, new features are correctly implemented (and tested): let's do a new release
- In production you have everything that works quite well, in development you have everything that works well with dummy data
- In staging you must test (everything again) with the real data, before to move in production
- Production and stage must have the same content, the same data and the same of most of configurations

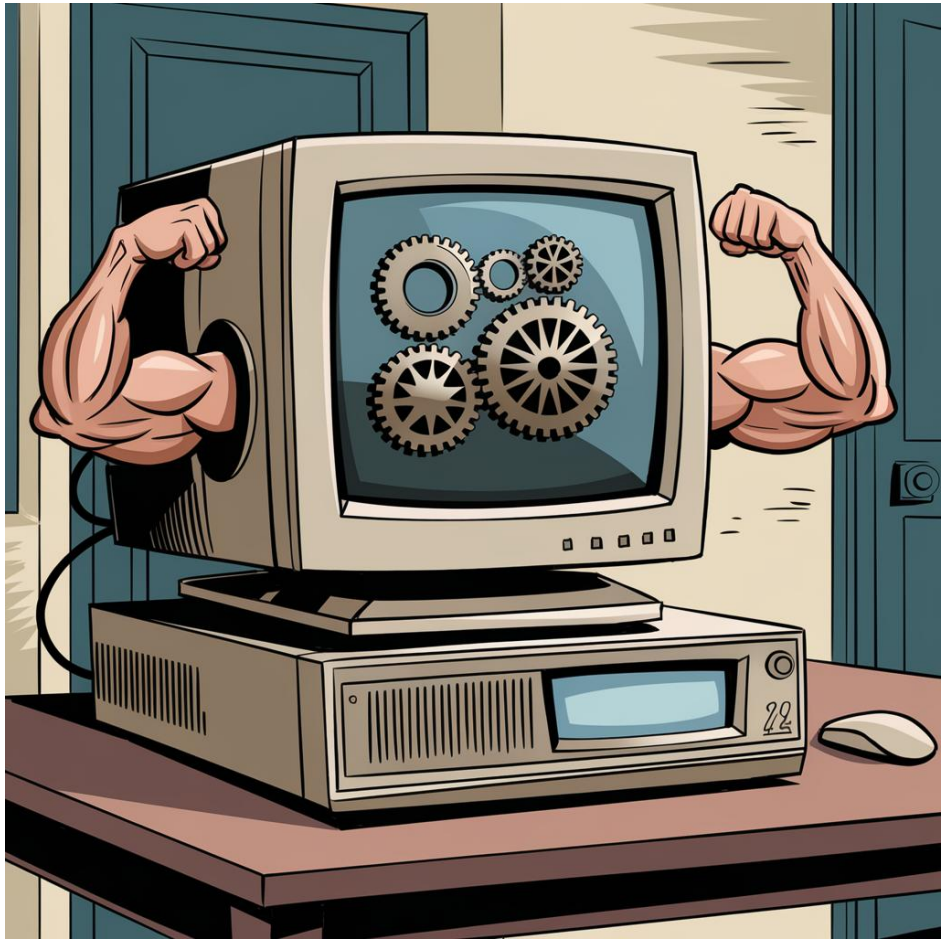
Staging environment

- In this environment, generally:
  - the customer or who express the business requirements tests the developments to check that everything works properly (a.k.a. User Acceptance Tests – UAT)
  - the data entry is performed in this environment and then replicated on Production
  - it's the last environment where you can deploy a release
  - you can perform a re-index of databases without an impact on the performance of production environment

Ca' Foscari University of Venice

Production environment

- This is the real environment, which all users will use
- Everything here must work properly, and the performance should be in line with the expectations
- You generally cannot perform a deployment but a replica from staging – obviously, depends on the choice that you (or others) taken



Benefits

- Ensure proper quality of software
- Share informations
- Monitor the processes
- Review and audit software products
- Ensure that software meets standards
- Ensure software matching with business requirements
- Involve entire software lifecycle
- Less rework
- Less time for final UAT
- More detailed documentation
- Higher quality of software release

Ca' Foscari University of Venice

and now let's try!

- TODO:
  - create your repo
  - create your machine on AWS/Azure
  - setup README.md file
  - write a simple application
  - save your work on your repo
    - open a PR and then merge it
  - deploy it on your machine
    - after you test it in your local environment
    - note: for this test you can use a different application stack
  - run it

- HINT
  - AI permits to you to save your time to write the simple application
  - use a language that you know, it helps
  - a simple webapp could be the simplest app to implement it
  - right now, an hello world is enough ;-)